

CONTROLLER ROUTER

Deployment Guide

Package the library and optional UNO Q runtime,
with safe upgrades and recovery.

VIRTUALGLOVE / R.O.B. VISION / MAKER CONTROLLERS

Current source edition | 29 September 2026

This guide covers packaging and deploying Controller Router inside your own controller project. The Integration Guide describes the design; the Technical Reference defines the APIs.

Install the console library

The repository's `pyproject.toml` packages `router_shared` as `maker-controller-router`. It declares Python 3.10 or newer. From a local checkout, install it in your project's virtual environment:

```
python3 -m venv .venv
. .venv/bin/activate
python3 -m pip install .
```

The library has no third-party Python runtime dependencies. Live routing requires Linux input devices, uinput permission, EmulationStation mappings for ordinary pads, and RetroArch. A macOS development machine can inspect and test the API but cannot create Linux uinput outputs.

The wheel is Python-only and does not need a CPU-specific build. It does not install service units, a receiver, game launch hooks, a web setup interface, or UNO Q firmware. Package those integration assets in your project.

Package a complete integration

1. Include the shared library through your dependency package or a maintained vendored copy.
2. Install your receiver and its stable uinput device definition.
3. Install any custom source descriptor and set `CONTROLLER_ROUTER_SOURCES_FILE` for the Router process.
4. Choose persistent assignment, backup, and socket paths appropriate to the console.
5. Import known EmulationStation mappings and add your custom source without deleting saved assignments.
6. Start `ControllerRouterDevice` with explicit platform configuration paths.
7. Expose bounded, authenticated setup operations through your project's service.
8. Install the session launch adapter, choose enabled systems, and test input in the actual game.

Provide service startup, shutdown, watchdog, permissions, and logs in your installer. Keep personal configuration separate from replaceable software. Avoid creating a second competing Router process when another compatible integration already owns the merged outputs.

Router recognizes RetroPie, Batocera, and Recalbox configuration conventions. That recognition alone does not validate your receiver, emulator wrapper, or installer on every board and version.

Protect console configuration

Finish games before installing or reconfiguring virtual controllers. Close EmulationStation on RetroPie before recreating virtual joysticks; its input manager can fail during device replacement.

Router's atomic writes preserve existing permissions and ownership. When writing as root under `/opt/retroPie/configs/`, the writer assigns `pi:pi`. Use that helper for managed writes rather than replacing a file through an unrelated root-owned temporary file.

Route mapping changes through the Router integration. Receivers and game-specific wrappers must not independently overwrite the same player indexes. Decide which platform files your adapter owns and document that boundary before deploying it.

Optional UNO Q runtime

The port-80 chooser links to **Setup** for Players and Systems and to **Pair console** on HTTPS port 8444. Router owns the console registry and authenticated console management connection. Product adapters retain game actions and scoped app credentials. Assignment changes take effect on the next launch.

The `uno_portal` directory supplies the browser chooser, host broker, product lifecycle service, lease coordinator, Matrix scheduler, and App Lab sketch. It is separate from the console wheel.

The current installer expects the `arduino` account with UID 1000, App Lab, Docker Compose, user services, and the existing registered product layout. It is designed for the current VirtualGlove and R.O.B. Vision adapters. A new app must extend that registration; this is not a generic install command for an arbitrary third product.

To add your app:

1. Add a fixed app ID and product path to the portal's registries and validation allowlists.
2. Define its browser port and local health endpoint.
3. Define how its authenticated game-session state becomes a live-game boolean.
4. Register its manifest path and include the display client in its runtime.
5. Add lease enforcement to its input producer and receiver.
6. Add the product to lifecycle startup and installer backup/upgrade handling.
7. Add its chooser entry and direct-visit selection behavior.
8. Extend tests before enabling it on a UNO Q.

Keep Router as the sole Matrix firmware owner. Product installation and startup should not flash a competing sketch. The shared Matrix's first App Lab build can take several minutes; keep progress visible and explain the wait to users.

Upgrade and rollback design

Store pairing credentials, game registries, and player assignments outside software replacement paths. Back up the current software before changing services. Verify new service and Matrix health; restore previous software if activation fails.

Keep a newer compatible shared Router when a product bundles an older copy, while still registering the new product. A newly installed controller must not disappear merely because shared code did not need upgrading.

Assignment rollback is separate from software rollback. RouterStore saves a `.previous` snapshot and requires a current revision for restoration. Your UI should explain which settings are being restored.

Existing integrations as deployment examples

R.O.B. Vision installs the shared console service and a fixed Buddy source descriptor, then keeps Buddy on Player 2 through its adapter. VirtualGlove vendors the same engine behind its signed console API. Both bundle `uno_portal` so their UNO Q installers register their services with the same Matrix owner.

Read the implementation before copying service paths or product assumptions:

- [R.O.B. Vision installer](#)
- [R.O.B. Vision Router setup](#)
- [VirtualGlove UNO Q installer](#)
- [Shared UNO installer](#)

Release checklist

Test a clean install, repeat upgrade, preserved assignments, failed-start rollback, input release on disconnect, and live-game configuration rejection. For UNO Q, also test no-selection reboot, automatic game selection, missing or expired leases, display expiry, and a newly registered app alongside a newer Router.

Record actual platform and device results. Keep build compatibility, mocked tests, and physical-device tests distinct in release notes.

Register session routing

Bundle `router_shared.launch` and `router_shared.launch_install` with the engine.

On RetroPie, call `launch_install.install_retropie()` during installation or upgrade. It backs up replaced Router files, repairs only Router-owned profiles, and registers the adapter in existing `emulators.cfg` commands. It does not edit saved `retroarch.cfg` files. Repeating installation leaves unchanged files alone.

On Batocera, call `install_batocera()` during installation and `activate_batocera()` during service startup. Installation validates the generator boundary before changing integration files. Startup binds the narrow generator overlay; it does not write saved RetroArch settings. Game hooks should report sessions only. A generation layout that cannot be identified stops installation.

For another launcher, invoke ``python3 -m router_shared.launch --config PATH --retroarch EXECUTABLE -- ORIGINAL_ARGUMENTS``. For preparation-only callers, ``controller_router prepare-launch --platform PLATFORM --config PATH --retroarch EXECUTABLE --output SESSION_FILE`` emits diagnostics and a temporary configuration.

Append that file last, keep it for the process lifetime, and delete it afterward.

The adapter handles that lifecycle, signals, exit status, and output-loss reporting.

Do not recreate merged outputs during play. Missing or duplicate configured outputs block a routed launch. A Router failure requires the player to exit and relaunch after recovery. Test legacy and reservation modes independently, including keyboard controls, hotkeys, and physical-source reconnects.

Preserve system policy during deployment

New format-2 configurations start with `physical_scope: "nes"`. Upgrades retain `nes`, `all`, or `systems` and any saved `physical_systems` list. Assignment-only requests from older clients preserve the existing policy.

The console adapter and input engine use the same canonical system ID. A disabled system receives no routing override and physical sources remain ungrabbed for that session. Keep merged output devices alive regardless of system policy. Standalone emulators bypass this integration.

Expose Players and Systems through your authenticated adapter, rather than writing configuration files from a browser. The [Technical Reference](TECHNICAL_REFERENCE.md#per-system-routing-policy) defines the fields; the [User Guide](USER_GUIDE.md) describes the Setup workflow.

RetroArch configuration precedence and upgrades

Router appends its temporary settings after existing appended files. RetroArch can still load a core or game override afterward. A user override containing joypad indexes or reservations can therefore replace the session routing. Preserve unrelated game overrides; inspect controller-specific overrides when a game selects the wrong merged player despite correct launch diagnostics.

During RetroPie installation or upgrade, Router backs up the FCEUmm and Nestopia core overrides and removes only `input_player1_joypad_index` through `input_player4_joypad_index` inside one recognized old Router block. Button bindings, hotkeys, unmarked indexes, and other settings remain. This migration is installer-only. Router never changes these files at receiver startup or game launch, and RetroPie configuration and profile files remain owned by `pi:pi`.

Shared device pairing

Controller Router is the connection authority. The UNO Q host broker runs secure Setup on TCP **8444** and stores schema-1 connections under `/home/arduino/.local/state/controller-router/connections.json`. The persistent console TLS service listens on TCP **55359**. Router has a management credential; VirtualGlove and R.O.B. Vision have distinct, independently revocable credentials. Browser responses contain only public connection and readiness fields.

`router_shared.pairing.Peer` checks the console certificate before sending a code or credential. The CR1 code contains a 100-bit certificate fingerprint prefix and a 60-bit authorization value. Console windows last 300 seconds, accept one successful transaction, and lock after five incorrect codes. Physical Matrix confirmation lasts 120 seconds and allows five attempts. Requests are bounded; servers allow at most 16 concurrent workers and require TLS 1.2 or later. Secure browser writes require matching HTTPS Origin, a fixed action header, and JSON content.

Provisioning uses prepare, commit, and finalize. Private journals retain the previous Router registry and app configuration; interrupted or failed transactions restore those snapshots. Successful changes keep private before-connection backups. Legacy migration verifies an HMAC over a fresh nonce, canonical console ID, and observed TLS certificate; adoption signs the new connection transcript. Hostnames alone never authorize consolidation. Conflicting records remain unchanged for an explicit re-pairing choice.

Products expose `/api/router-pairing` only to a capability-bearing local host request. Capability files are named `data/router-pairing-adapter-token` and must not be exposed to browsers. Adapters import app credentials into live caches without replacing ROM registries, calibration, or player settings. Router polls for newly installed console and UNO adapters and provisions them using its pinned management connection. Disabled app access stays disabled.

The console helper has `/identity`, `/pair`, `/legacy-proof`, `/adopt`, `/manage`, and `/router` interfaces. `/manage` uses Router's credential for inspection, provisioning, and removal. `/router` uses the same credential with RouterStore revision checks for assignments and system policy. It never writes saved RetroArch configurations. Buddy's first-pair adapter adds only its Player 2 source to Router configuration; emulator configuration migration remains installation-only.

Back up the UNO Q connection registry, its `tls` directory, and the private before-connection backups. On RetroPie, also back up `/var/lib/controller-router/link/{console-id, adapters.json, connection.json, certificate.pem, private-key.pem}` and each product's credential files. Restore requires the separately installed shared link service. Keep backups private; never include credentials in diagnostics or support reports. Do not snapshot a provisioning transaction in progress.